

Change the billing state of a subscription for a user

Article • 10/20/2022

Use this method in the Microsoft Store purchase API to change the billing state of a subscription add-on for a given user. You can cancel, extend, refund, or disable automatic renewal for a subscription.

ⓘ Note

This method can only be used by developer accounts that have been provisioned by Microsoft to be able to create subscription add-ons for Universal Windows Platform (UWP) apps. Subscription add-ons are currently not available to most developer accounts.

The [Microsoft.StoreServices library](#) provides the functionality of this method through the `StoreServicesClient.RecurrenceChangeAsync` API.

Prerequisites

To use this method, you will need:

- An Azure AD access token that has the audience URI value `https://onestore.microsoft.com`.
- A Microsoft Store ID key that represents the identity of the user who has an entitlement to the subscription you want to change.

For more information, see [Manage product entitlements from a service](#).

Request

Request syntax

Method	Request URI
POST	<code>https://purchase.mp.microsoft.com/v8.0/b2b/recurrences/{recurrenceId}/change</code>

Request header

Header	Type	Description
Authorization	string	Required. The Azure AD access token in the form Bearer <i><token></i> .
Host	string	Must be set to the value purchase.mp.microsoft.com .
Content-Length	number	The length of the request body.
Content-Type	string	Specifies the request and response type. Currently, the only supported value is application/json .

Request parameters

Name	Type	Description	Required
recurrenceId	string	The ID of the subscription you want to change. To get this ID, call the get subscriptions for a user method, identify the response body entry that represents the subscription add-on you want to change, and use the value of the id field for the entry.	Yes

Request body

Field	Type	Description	Required
b2bKey	string	The Microsoft Store ID key that represents the identity of the user whose subscription you want to change.	Yes
changeType	string	One of the following strings that identifies the type of change you want to make: Cancel: Cancels the subscription. Extend: Extends the subscription. If you specify this value, you must also include the <i>extensionTimeInDays</i> parameter. Refund: Refunds the subscription to the customer. ToggleAutoRenew: Disables automatic renewal for the subscription. If automatic renewal is currently disabled for the subscription, this value does nothing.	Yes

Field	Type	Description	Required
extensionTimeInDays	string	If the <i>changeType</i> parameter has the value Extend , this parameter specifies the number of days to extend the subscription.	Yes, if <i>changeType</i> has the value Extend ; otherwise, no.

Request example

The following example demonstrates how to use this method to extend the subscription period by 5 days. Replace the *b2bKey* value with the [Microsoft Store ID key](#) that represents the identity of the user whose subscription you want to change.

JSON

```
POST
https://purchase.mp.microsoft.com/v8.0/b2b/recurrences/mdr:0:bc0cb6960acd4515a0e1d638192d77b7:77d5ebec-0310-4d23-b204-83e8613baaac/change HTTP/1.1
Authorization: Bearer <your access token>
Content-Type: application/json
Host: https://purchase.mp.microsoft.com

{
  "b2bKey": "eyJ0eXAiOiJ...",
  "changeType": "Extend",
  "extensionTimeInDays": "5"
}
```

Response

This method returns a JSON response body that contains information about the subscription add-on that was modified, including any fields that were modified. The following example demonstrates a response body for this method.

JSON

```
{
  "items": [
    {
      "autoRenew": true,
      "beneficiary": "pub:gFVuEBiZHPXonkYvtdOi+tLE2h4g2Ss0ZId0RQOwzDg=",
      "expirationTime": "2017-06-16T03:07:49.2552941+00:00",
      "id": "mdr:0:bc0cb6960acd4515a0e1d638192d77b7:77d5ebec-0310-4d23-b204-83e8613baaac",
      "lastModified": "2017-01-10T21:08:13.1459644+00:00",
      "market": "US",
    }
  ]
}
```

```
"productId": "9NBLGGH52Q8X",
"skuId": "0024",
"startTime": "2017-01-10T21:07:49.2552941+00:00",
"recurrenceState": "Active"
}
]
}
```

Response body

The response body contains the following data.

Value	Type	Description
autoRenew	Boolean	Indicates whether the subscription is configured to automatically renew at the end of the current subscription period.
beneficiary	string	The ID of the beneficiary of the entitlement that is associated with this subscription.
expirationTime	string	The date and time the subscription will expire, in ISO 8601 format. This field is only available when the subscription is in certain states. The expiration time usually indicates when the current state expires. For example, for an active subscription, the expiration date indicates when the next automatic renewal will occur.
expirationTimeWithGrace	string	The date and time the subscription will expire including the grace period, in ISO 8601 format. This value indicates when the user will lose access to the subscription after the subscription has failed to automatically renew.
id	string	The ID of the subscription. Use this value to indicate which subscription you want to modify when you call the change the billing state of a subscription for a user method.
isTrial	Boolean	Indicates whether the subscription is a trial.
lastModified	string	The date and time the subscription was last modified, in ISO 8601 format.
market	string	The country code (in two-letter ISO 3166-1 alpha-2 format) in which the user acquired the subscription.
productId	string	The Store ID for the product that represents the subscription add-on in the Microsoft Store catalog. An example Store ID for a product is 9NBLGGH42CFD.

Value	Type	Description
skuld	string	The Store ID for the SKU that represents the subscription add-on the Microsoft Store catalog. An example Store ID for a SKU is 0010.
startTime	string	The start date and time for the subscription, in ISO 8601 format.

Value	Type	Description
recurrenceState	string	One of the following values: • None: This indicates a perpetual subscription. • Active: The subscription is active and the user is entitled to use the services. • Inactive: The subscription is past the expiration date, and the user turned off the automatic renew option for the subscription. • Canceled: The subscription has been purposefully terminated before the expiration date, with or without a refund. • InDunning: The subscription is in <i>dunning</i> (that is, the subscription is nearing expiration, and Microsoft is trying to acquire funds to automatically renew the subscription). • Failed: The dunning period is over and the subscription failed to renew after several attempts. Note: • Inactive/Canceled/Failed are terminal states. When a subscription enters one of these states, the user must repurchase the subscription to activate it again. The user is not entitled to use the services in these states. • When a subscription is Canceled, the <code>expirationTime</code> will be updated with the date and time of the cancellation. • The ID of the subscription will remain the same during its entire lifetime. It will not change if the auto-renew option is turned on or off. If a user repurchases a subscription after reaching a terminal state, a new subscription ID will be created. • The ID of a subscription should be used to execute any operation on an individual subscription. • When a user repurchases a subscription after cancelling or discontinuing it, if you query the results for the user you will get two entries: one with the old subscription ID in a terminal state, and one with the new subscription ID in an active state. • It's always a good practice to check both <code>recurrenceState</code> and <code>expirationTime</code>, since updates to <code>recurrenceState</code> can potentially be delayed by a few minutes (or occasionally hours).
cancellationDate	string	The date and time the user's subscription was cancelled, in ISO 8601 format.

Related topics

- [Manage product entitlements from a service](#)
- [Get subscriptions for a user](#)
- [Query for products](#)
- [Report consumable products as fulfilled](#)
- [Renew a Microsoft Store ID key](#)
- [Microsoft.StoreServices library \(GitHub\) !\[\]\(6841ca9b0e023296428e7c9e683b9367_img.jpg\)](#)

Renew a Microsoft Store ID key

Article • 12/11/2024

Use this method to renew a Microsoft Store key. When you [generate a Microsoft Store ID key](#), the key is valid for 30 days. Before the key expires, you can use it to renegotiate a new key by using this method.

The [Microsoft.StoreServices library](#) [↗] provides the functionality of this method through the UserStoreId.RefreshStoreId API.

ⓘ Note

Previously Microsoft Store keys had a longer lifetime and in some cases were able to be renewed after the key had already expired. Now, developers should only expect keys to be renewable during their lifetime and should note the expire date on the keys in order to renew them before they expire.

ⓘ Important

Although Microsoft Store keys are able to be renewed before they expire, the keys may be revoked prior to their expire dates in response to security concerns. Therefore, your services should handle the case where a renewal attempt fails with the AuthenticationTokenInvalid. In this case, you should generate a new Microsoft Store key for the user on the client or the method used to generate the original Microsoft Store key.

Prerequisites

To use this method, you will need:

- An Entra ID access token that has the audience URI value `https://onestore.microsoft.com/.default`.
- An expired Microsoft Store ID key that was [generated from client-side code in your app](#).

For more information, see [Manage product entitlements from a service](#).

Request

Request syntax

 Expand table

Key type	Method	Request URI
Collections	POST	<code>https://collections.mp.microsoft.com/v6.0/b2b/keys/renew</code>
Purchase	POST	<code>https://purchase.mp.microsoft.com/v6.0/b2b/keys/renew</code>

Request header

 Expand table

Header	Type	Description
Host	string	Must be set to the value <code>collections.mp.microsoft.com</code> or <code>purchase.mp.microsoft.com</code> .
Content-Length	number	The length of the request body.
Content-Type	string	Specifies the request and response type. Currently, the only supported value is <code>application/json</code> .

Request body

 Expand table

Parameter	Type	Description	Required
serviceTicket	string	The Entra ID access token.	Yes
key	string	The expired Microsoft Store ID key.	Yes

Request example

syntax

```
POST https://collections.mp.microsoft.com/v6.0/b2b/keys/renew HTTP/1.1
Content-Length: 2774
Content-Type: application/json
Host: collections.mp.microsoft.com

{
```

```
"serviceTicket": "eyJ0eXAiOiJKV1QiLCJhbG...",
"Key": "eyJ0eXAiOiJKV1QiLCJhbG..."
}
```

Response

Response body

 Expand table

Parameter	Type	Description
key	string	The refreshed Microsoft Store key that can be used in future calls to the Microsoft Store collections API or purchase API.

Response example

syntax

```
HTTP/1.1 200 OK
Content-Length: 1646
Content-Type: application/json
MS-CorrelationId: aaaa0000-bb11-2222-33cc-444444444444
MS-RequestId: 1b5fa630-d672-4971-b2c0-3713f4ea6c85
MS-CV: xu2HW6SrSkyfHyFh.0.0
MS-ServerId: 030011428
Date: Tue, 13 Sep 2015 07:31:12 GMT

{
  "key": "eyJ0eXAiOiJKV1QiLCJhbG..."
}
```

Error codes

 Expand table

Code	Error	Inner error code	Description
401	Unauthorized	AuthenticationTokenInvalid	The Entra ID access token or the Microsoft Store Key being renewed are invalid. In some cases the details of the ServiceError will contain more information, such as when

Code	Error	Inner error code	Description
			the token is expired or the <i>appid</i> claim is missing.
401	Unauthorized	InconsistentClientId	The <i>clientId</i> claim in the Microsoft Store ID key and the <i>appid</i> claim in the Entra ID access token do not match.

Related topics

- [Manage product entitlements from a service](#)
- [Query for products](#)
- [Report consumable products as fulfilled](#)
- [Grant free products](#)
- [Microsoft.StoreServices library \(GitHub\)](#) 

Feedback

Was this page helpful? 👍 Yes 👎 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Send requests to the Microsoft Store

Article • 07/16/2024

Starting in Windows 10, version 1607, the Windows SDK provides APIs for Store-related operations (such as in-app purchases) in the [Windows.Services.Store](#) namespace. However, although the services that support the Store are constantly being updated, expanded, and improved between OS releases, new APIs are typically added to the Windows SDK only during major OS releases.

We provide the [SendRequestAsync](#) method as a flexible way to make new Store operations available to Universal Windows Platform (UWP) apps before a new version of the Windows SDK is released. You can use this method to send requests to the Store for new operations that do not yet have a corresponding API available in the latest release of the Windows SDK.

ⓘ Note

The **SendRequestAsync** method is available only to apps that target Windows 10, version 1607, or later. Some of the requests supported by this method are only supported in releases after Windows 10, version 1607.

SendRequestAsync is a static method of the [StoreRequestHelper](#) class. To call this method, you must pass the following information to the method:

- A [StoreContext](#) object that provides information about the user for which you want to perform the operation. For more information about this object, see [Get started with the StoreContext class](#).
- An integer that identifies the request that you want to send to the Store.
- If the request supports any arguments, you can also pass a JSON-formatted string that contains the arguments to pass along with the request.

The following example demonstrates how to call this method. This example requires using statements for the **Windows.Services.Store** and **System.Threading.Tasks** namespaces.

C#

```
public async Task<bool> AddUserToFlightGroup()
{
    StoreSendRequestResult result = await
    StoreRequestHelper.SendRequestAsync(
        StoreContext.GetDefault(), 8,
```

```

        "{ \"type\": \"AddToFlightGroup\", \"parameters\": {
        \"flightGroupId\": \"your group ID\" } }");

        if (result.ExtendedError == null)
        {
            return true;
        }

        return false;
    }

```

See the following sections for information about the requests that are currently available via the **SendRequestAsync** method. We will update this article when support for new requests are added.

Request for in-app ratings and reviews

You can programmatically launch a dialog from your app that asks your customer to rate your app and submit a review by passing the request integer 16 to the **SendRequestAsync** method. For more information, see [Show a rating and review dialog in your app](#).

Requests for flight group scenarios

Important

All of the flight group requests described in this section are currently not available to most developer accounts. These requests will fail unless your developer account is specially provisioned by Microsoft.

The **SendRequestAsync** method supports a set of requests for flight group scenarios, such as adding a user or device to a flight group. To submit these requests, pass the value 7 or 8 to the *requestKind* parameter along with a JSON-formatted string to the *parametersAsJson* parameter that indicates the request you want to submit along with any related arguments. The method also supports requests to map a PFN with a BigID (ProductID from Display Catalog). To submit this request, pass the value 9 to the *requestKind* parameter. These *requestKind* values differ in the following ways.

 Expand table

Request kind value	Description
7	The requests are performed in the context of the current device. This value can only be used on Windows 10, version 1703, or later.
8	The requests are performed in the context of the user who is currently signed in to the Store. This value can be used on Windows 10, version 1607, or later.
9	The requests return the BigID (ProductId) by PackageFamilyName via an HTTP request to the Display Catalog with anonymous authentication.

The following flight group requests are currently implemented.

Retrieve remote variables for the highest-ranked flight group

Important

This request is currently not available to most developer accounts. This request will fail unless your developer account is specially provisioned by Microsoft.

This request retrieves the remote variables for the highest-ranked flight group for the current user or device. To send this request, pass the following information to the *requestKind* and *parametersAsJson* parameters of the **SendRequestAsync** method.

 Expand table

Parameter	Description
<i>requestKind</i>	Specify 7 to return the highest-ranked flight group for the device, or specify 8 to return the highest-ranked flight group for the current user and device. We recommend using the value 8 for the <i>requestKind</i> parameter, because this value will return the highest-ranked flight group across the membership for both the current user and device.
<i>parametersAsJson</i>	Pass a JSON-formatted string that contains the data shown in the example below.

The following example shows the format of the JSON data to pass to *parametersAsJson*. The *type* field must be assigned to the string *GetRemoteVariables*. Assign the *projectId* field to the ID of the project in which you defined the remote variables in Partner Center.

JSON

```
{
  "type": "GetRemoteVariables",
  "parameters": "{ \"projectId\": \"your project ID\" }"
}
```

After you submit this request, the [Response](#) property of the [StoreSendRequestResult](#) return value contains a JSON-formatted string with the following fields.

[Expand table](#)

Field	Description
<i>anonymous</i>	A Boolean value, where true indicates that the user or device identity was not present in the request, and false indicates that user or device identity was present in the request.
<i>name</i>	A string that contains the name of the highest-ranked flight group to which the device or user belongs.
<i>settings</i>	A dictionary of key/value pairs that contain the name and value of the remote variables that the developer has configured for the flight group.

The following example demonstrates a return value for this request.

JSON

```
{
  "anonymous": false,
  "name": "Insider Slow",
  "settings":
  {
    "Audience": "Slow"
    ...
  }
}
```

Add the current device or user to a flight group

Important

This request is currently not available to most developer accounts. This request will fail unless your developer account is specially provisioned by Microsoft.

To send this request, pass the following information to the *requestKind* and *parametersAsJson* parameters of the **SendRequestAsync** method.

 Expand table

Parameter	Description
<i>requestKind</i>	Specify 7 to add the device to a flight group, or specify 8 to add the user who is currently signed in to the Store to a flight group.
<i>parametersAsJson</i>	Pass a JSON-formatted string that contains the data shown in the example below.

The following example shows the format of the JSON data to pass to *parametersAsJson*. The *type* field must be assigned to the string *AddToFlightGroup*. Assign the *flightGroupId* field to the ID of the flight group to which you want to add the device or user.

JSON

```
{
  "type": "AddToFlightGroup",
  "parameters": "{ \"flightGroupId\": \"your group ID\" }"
}
```

If there is an error with the request, the [HttpStatusCode](#) property of the [StoreSendRequestResult](#) return value contains the response code.

Remove the current device or user from a flight group

Important

This request is currently not available to most developer accounts. This request will fail unless your developer account is specially provisioned by Microsoft.

To send this request, pass the following information to the *requestKind* and *parametersAsJson* parameters of the **SendRequestAsync** method.

 Expand table

Parameter	Description
<i>requestKind</i>	Specify 7 to remove the device from a flight group, or specify 8 to remove the user who is currently signed in to the Store from a flight group.
<i>parametersAsJson</i>	Pass a JSON-formatted string that contains the data shown in the example below.

The following example shows the format of the JSON data to pass to *parametersAsJson*. The *type* field must be assigned to the string *RemoveFromFlightGroup*. Assign the *flightGroupId* field to the ID of the flight group from which you want to remove the device or user.

JSON

```
{
  "type": "RemoveFromFlightGroup",
  "parameters": "{ \"flightGroupId\": \"your group ID\" }"
}
```

If there is an error with the request, the [HttpStatusCode](#) property of the [StoreSendRequestResult](#) return value contains the response code.

Related topics

- [Show a rating and review dialog in your app](#)
- [SendRequestAsync](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Add retail demo (RDX) features to your app

Article • 10/20/2022

Include a retail demo mode in your Windows app so customers who try out PCs and devices on the sales floor can jump right in.

When customers are in a retail store, they expect to be able to try out demos of PCs and devices. They often spend a considerable chunk of their time playing around with apps through the [retail demo experience \(RDX\)](#).

You can set up your app to provide different experiences while in *normal* or *retail* modes. For example, if your app starts with a setup process, you might skip past it in retail mode, and prepopulate the app with sample data and default settings so they can jump right in.

From our customers' perspective, there is only one app. To help customers distinguish between the two modes, we recommend that while your app is in retail mode, it shows the word "Retail" prominently in the title bar or in a suitable location.

In addition to the Microsoft Store requirements for apps, RDX-aware apps must also be compatible with the RDX setup, cleanup, and update processes to ensure that customers have a consistently positive experience at the retail store.

Design principles

- **Show your best.** Use the retail demo experience to showcase why your app rocks. This is likely the first time your customer will see your app, so show them the best piece!
- **Show it fast.** Customers can be impatient - The faster a user can experience the real value of your app, the better.
- **Keep the story simple.** The retail demo experience is an elevator pitch for your app's value.
- **Focus on the experience.** Give the user time to digest your content. While getting them to the best part fast is important, designing suitable pauses can help them to fully enjoy the experience.

Technical requirements

As RDX-aware apps are meant to showcase the best of your app to retail customers, they must meet technical requirements and adhere to privacy regulations that the Microsoft Store has for all retail demo experience apps.

This can be used as a checklist to help you prepare for the validation process and to provide clarity in the testing process. Note that these requirements have to be maintained, not just for the validation process, but for the entire lifetime of the retail demo experience app; as long as your app stays running on the retail demo devices.

Critical requirements

RDX-aware apps that do not meet these critical requirements will be removed from all retail demo devices as soon as possible.

- **Don't ask for personal identifiable information (PII).** This includes login info, Microsoft account info, or contact details.
- **Error-free experience.** Your app must run with no errors. Additionally, no error pop ups or notifications should be shown to customers using the retail demo devices. Errors reflect negatively on the app itself, your brand, the device's brand, the device's manufacturer's brand, and Microsoft's brand.
- **Paid apps must have a trial mode.** Your app either needs to be a free or include a [trial mode](#). Customers aren't looking to pay for an experience in a retail store.

High-priority requirements

RDX-aware apps that do not meet these high priority requirements need to be investigated for a fix immediately. If no immediate fix is found, this app may be removed from all retail demo devices.

- **Memorable offline experience.** Your app needs to demonstrate a great offline experience as about 50% of the devices are offline at retail locations. This is to ensure that customers interacting with your app offline are still able to have a meaningful and positive experience.
- **Updated content experience.** Your app should never be prompt for updates when online. If updates are needed, they should be performed silently.
- **No anonymous communication.** Because a customer using a retail demo device is an anonymous user, they should not be able to message or share content from the

device.

- **Deliver consistent experiences by using the cleanup process.** Every customer should have the same experience when they walk up to a retail demo device. Your app should use [clean up process](#) to return to the same default state after each use. We don't want the next customer to see what the last customer left behind. This includes scoreboards, achievements, and unlocks.
- **Age appropriate content.** All app content needs to be assigned a Teen or lower rating category. To learn more, see [Getting your app rated by IARC](#) and [ESRB ratings](#).

Medium-priority requirements

The Windows Retail Store team may reach out to developers directly to set up a discussion on how to fix these issues.

- **Ability to run successfully over a range of devices.** Apps must run well on all devices, including devices with low-end specifications. If the app is installed on devices that did not meet the minimum specifications, the app needs to clearly inform the user about this. Minimum device requirements has to be made known so that the app can always run with high performance.
- **Meet retail store app size requirements.** The app must be smaller than 800MB. Contact the Windows Retail Store team directly for further discussion if your RDX-aware app does not meet the size requirements.

RetailInfo API: Preparing your code for demo mode

IsDemoModeEnabled

The [IsDemoModeEnabled](#) property in the [RetailInfo](#) utility class, which is part of the [Windows.System.Profile](#) namespace in the Windows 10 and Windows 11 SDK, is used as a Boolean indicator to specify which code path your app runs on - the *normal* mode or the *retail* mode.

C#

```
using Windows.Storage;

StorageFolder folder = ApplicationData.Current.LocalFolder;
```

```

if (Windows.System.Profile.RetailInfo.IsDemoModeEnabled)
{
    // Use the demo specific directory
    folder = await folder.GetFolderAsync("demo");
}

StorageFile file = await folder.GetFilesAsync("hello.txt");
// Now read from file

```

C++

```

using namespace Windows::Storage;

StorageFolder^ localFolder = ApplicationData::Current->LocalFolder;

if (Windows::System::Profile::RetailInfo::IsDemoModeEnabled)
{
    // Use the demo specific directory
    create_task(localFolder->GetFolderAsync("demo").then([this]
(StorageFolder^ demoFolder)
    {
        return demoFolder->GetFileAsync("hello.txt");
    }).then([this](task<StorageFile^> fileTask)
    {
        StorageFile^ file = fileTask.get();
    }));
    // Do something with file
}
else
{
    create_task(localFolder->GetFileAsync("hello.txt").then([this]
(StorageFile^ file)
    {
        // Do something with file
    }));
}

```

JavaScript

```

if (Windows.System.Profile.retailInfo.isDemoModeEnabled) {
    console.log("Retail mode is enabled.");
} else {
    Console.log("Retail mode is not enabled.");
}

```

RetailInfo.Properties

When `IsDemoModeEnabled` returns true, you can query for a set of properties about the device using `RetailInfo.Properties` to build a more customized retail demo experience. These properties include `ManufacturerName`, `ScreenSize`, `Memory` and so on.

C#

```
using Windows.UI.Xaml.Controls;
using Windows.System.Profile;

TextBlock priceText = new TextBlock();
priceText.Text = RetailInfo.Properties[KnownRetailInfo.Price];
// Assume infoPanel is a StackPanel declared in XAML
this.infoPanel.Children.Add(priceText);
```

C++

```
using namespace Windows::UI::Xaml::Controls;
using namespace Windows::System::Profile;

TextBlock ^manufacturerText = ref new TextBlock();
manufacturerText.set_Text(RetailInfo::Properties[KnownRetailInfoProperties::Price]);
// Assume infoPanel is a StackPanel declared in XAML
this->infoPanel->Children->Add(manufacturerText);
```

JavaScript

```
var pro = Windows.System.Profile;
console.log(pro.retailInfo.properties[pro.KnownRetailInfoProperties.price]);
```

IDL

C++

```
// Copyright (c) Microsoft Corporation. All rights reserved.
//
// WindowsRuntimeAPISet

import "oaidl.idl";
import "inspectable.idl";
import "Windows.Foundation.idl";
#include <sdkddkver.h>

namespace Windows.System.Profile
{
    runtimeclass RetailInfo;
    runtimeclass KnownRetailInfoProperties;
```

```

[version(NTDDI_WINTHRESHOLD), uuid(0712C6B8-8B92-4F2A-8499-
031F1798D6EF), exclusiveto(RetailInfo)]
[version(NTDDI_WINTHRESHOLD, Platform.WindowsPhone)]
interface IRetailInfoStatics : IInspectable
{
    [propget] HRESULT IsDemoModeEnabled([out, retval] boolean *value);
    [propget] HRESULT Properties([out, retval, hasvariant]
Windows.Foundation.Collections.IMapView<HSTRING, IInspectable *> **value);
}

[version(NTDDI_WINTHRESHOLD), uuid(50BA207B-33C4-4A5C-AD8A-
CD39F0A9C2E9), exclusiveto(KnownRetailInfoProperties)]
[version(NTDDI_WINTHRESHOLD, Platform.WindowsPhone)]
interface IKnownRetailInfoPropertiesStatics : IInspectable
{
    [propget] HRESULT RetailAccessCode([out, retval] HSTRING *value);
    [propget] HRESULT ManufacturerName([out, retval] HSTRING *value);
    [propget] HRESULT ModelName([out, retval] HSTRING *value);
    [propget] HRESULT DisplayModelName([out, retval] HSTRING *value);
    [propget] HRESULT Price([out, retval] HSTRING *value);
    [propget] HRESULT IsFeatured([out, retval] HSTRING *value);
    [propget] HRESULT FormFactor([out, retval] HSTRING *value);
    [propget] HRESULT ScreenSize([out, retval] HSTRING *value);
    [propget] HRESULT Weight([out, retval] HSTRING *value);
    [propget] HRESULT DisplayDescription([out, retval] HSTRING *value);
    [propget] HRESULT BatteryLifeDescription([out, retval] HSTRING
*value);
    [propget] HRESULT ProcessorDescription([out, retval] HSTRING
*value);
    [propget] HRESULT Memory([out, retval] HSTRING *value);
    [propget] HRESULT StorageDescription([out, retval] HSTRING *value);
    [propget] HRESULT GraphicsDescription([out, retval] HSTRING *value);
    [propget] HRESULT FrontCameraDescription([out, retval] HSTRING
*value);
    [propget] HRESULT RearCameraDescription([out, retval] HSTRING
*value);
    [propget] HRESULT HasNfc([out, retval] HSTRING *value);
    [propget] HRESULT HasSdSlot([out, retval] HSTRING *value);
    [propget] HRESULT HasOpticalDrive([out, retval] HSTRING *value);
    [propget] HRESULT IsOfficeInstalled([out, retval] HSTRING *value);
    [propget] HRESULT WindowsVersion([out, retval] HSTRING *value);
}

[version(NTDDI_WINTHRESHOLD), static(IRetailInfoStatics,
NTDDI_WINTHRESHOLD)]
[version(NTDDI_WINTHRESHOLD, Platform.WindowsPhone),
static(IRetailInfoStatics, NTDDI_WINTHRESHOLD, Platform.WindowsPhone)]
[threading(both)]
[marshaling_behavior(agile)]
runtimeclass RetailInfo
{
}

[version(NTDDI_WINTHRESHOLD), static(IKnownRetailInfoPropertiesStatics,

```

```

NTDDI_WINTHRESHOLD)]
    [version(NTDDI_WINTHRESHOLD, Platform.WindowsPhone),
    static(IKnownRetailInfoPropertiesStatics, NTDDI_WINTHRESHOLD,
    Platform.WindowsPhone)]
    [threading(both)]
    [marshaling_behavior(agile)]
    runtimeclass KnownRetailInfoProperties
    {
    }
}

```

Cleanup process

Cleanup begins two minutes after a shopper stops interacting with the device. The retail demo plays, and Windows begins resetting any sample data in the contacts, photos, and other apps. Depending on the device, this could take between 1-5 minutes to fully reset everything back to normal. This ensures that every customer in the retail store can walk up to a device and have the same experience when interacting with the device.

Step 1: Cleanup

- All Win32 and store apps are closed
- All files in known folders like **Pictures**, **Videos**, **Music**, **Documents**, **SavedPictures**, **CameraRoll**, **Desktop** and **Downloads** folders are deleted
- Unstructured and structured roaming states are deleted
- Structured local states are deleted

Step 2: Setup

- For offline devices: Folders remain empty
- For online devices: Retail demo assets can be pushed to the device from the Microsoft Store

Store data across user sessions

To store data across user sessions, you can store information in **ApplicationData.Current.TemporaryFolder** as the default cleanup process does not automatically delete data in this folder. Note that information stored using *LocalState* is deleted during the cleanup process.

Customize the cleanup process

To customize the cleanup process, implement the `Microsoft-RetailDemo-Cleanup` app service into your app.

Scenarios where a custom cleanup logic is needed includes running an extensive setup, downloading and caching data, or not wanting *LocalState* data to be deleted.

Step 1: Declare the *Microsoft-RetailDemo-Cleanup* service in your app manifest.

CSharp

```
<Applications>
  <Extensions>
    <uap:Extension Category="windows.appService"
EntryPoint="MyCompany.MyApp.RDXCustomCleanupTask">
      <uap:AppService Name="Microsoft-RetailDemo-Cleanup" />
    </uap:Extension>
  </Extensions>
</Application>
</Applications>
```

Step 2: Implement your custom cleanup logic under the *AppdataCleanup* case function using the sample template below.

CSharp

```
using System;
using System.IO;
using System.Runtime.Serialization.Json;
using System.Threading;
using System.Threading.Tasks;
using Windows.ApplicationModel.AppService;
using Windows.ApplicationModel.Background;
using Windows.Foundation.Collections;
using Windows.Storage;

namespace MyCompany.MyApp
{
    public sealed class RDXCustomCleanupTask : IBackgroundTask
    {
        BackgroundTaskCancellationReason _cancelReason =
BackgroundTaskCancellationReason.Abort;
        BackgroundTaskDeferral _deferral = null;
        IBackgroundTaskInstance _taskInstance = null;
        AppServiceConnection _appServiceConnection = null;

        const string MessageCommand = "Command";

        public void Run(IBackgroundTaskInstance taskInstance)
        {
            // Get the deferral object from the task instance, and take a
```

```

reference to the taskInstance;
    _deferral = taskInstance.GetDeferral();
    _taskInstance = taskInstance;
    _taskInstance.Canceled += new
BackgroundTaskCanceledEventHandler(OnCanceled);

    AppServiceTriggerDetails appService =
_taskInstance.TriggerDetails as AppServiceTriggerDetails;
    if ((appService != null) && (appService.Name == "Microsoft-
RetailDemo-Cleanup"))
    {
        _appServiceConnection = appService.AppServiceConnection;
        _appServiceConnection.RequestReceived +=
_appServiceConnection_RequestReceived;
        _appServiceConnection.ServiceClosed +=
_appServiceConnection_ServiceClosed;
    }
    else
    {
        _deferral.Complete();
    }
}

void _appServiceConnection_ServiceClosed(AppServiceConnection
sender, AppServiceClosedEventArgs args)
{
}

async void
_appServiceConnection_RequestReceived(AppServiceConnection sender,
AppServiceRequestReceivedEventArgs args)
{
    //Get a deferral because we will be calling async code
    AppServiceDeferral requestDeferral = args.GetDeferral();
    string command = null;
    var returnData = new ValueSet();

    try
    {
        ValueSet message = args.Request.Message;
        if (message.ContainsKey(MessageCommand))
        {
            command = message[MessageCommand] as string;
        }

        if (command != null)
        {
            switch (command)
            {
                case "AppdataCleanup":
                {
                    // Do custom clean up logic here
                    break;
                }
            }
        }
    }
}

```

```

        }
    }
    catch (Exception e)
    {
    }
    finally
    {
        requestDeferral.Complete();
        // Also release the task deferral since we only process one
        request per instance.
        _deferral.Complete();
    }
}

private void OnCanceled(IBackgroundTaskInstance sender,
BackgroundTaskCancellationReason reason)
{
    _cancelReason = reason;
}
}
}

```

Related links

- [Store and retrieve app data](#)
- [How to create and consume an app service](#)
- [Localizing app contents](#)
- [Retail demo experience \(RDX\)](#)

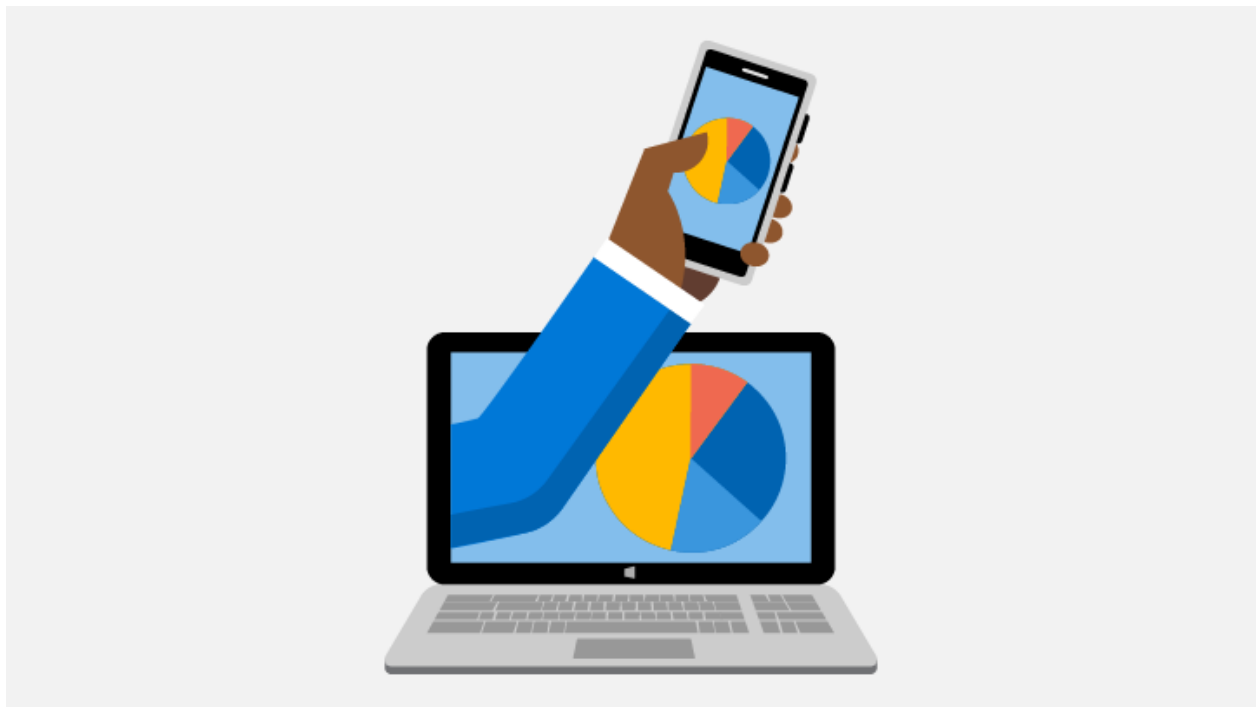
Monetization, engagement, and Store services

Article • 06/19/2023

The Windows SDK, Microsoft Advertising SDK, Microsoft Store Services SDK, and the Microsoft Store provide features that enable you to make more money from your apps and gain customers by engaging with your users. The topics in this section show you how to build these features into your app.

For details about the fees charged by the Microsoft Store and how you get paid for the money your app makes, see [Getting paid](#).

Choose a pricing model



Charge a price for your app

You can charge a price for your app up front. We support a comprehensive range of price tiers, including the option to set per-market prices. You can even schedule a sale to reduce the price of your app for a limited period of time.

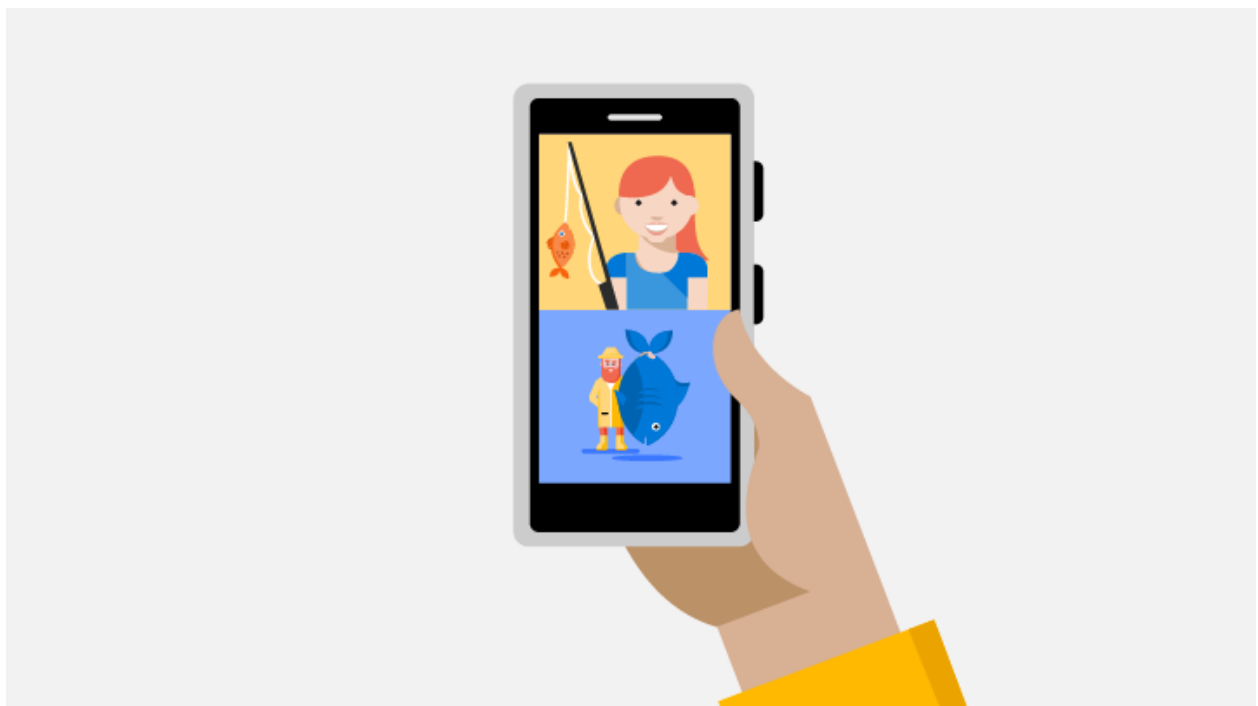
[Set app pricing](#)



Free trials

You can offer a free trial version of your app to get more customers to try it. To entice customers to buy the full version, you can limit the features in the trial version (for example, only including the first level of a game), show ads, or specify a time-limited trial.

Offer a trial

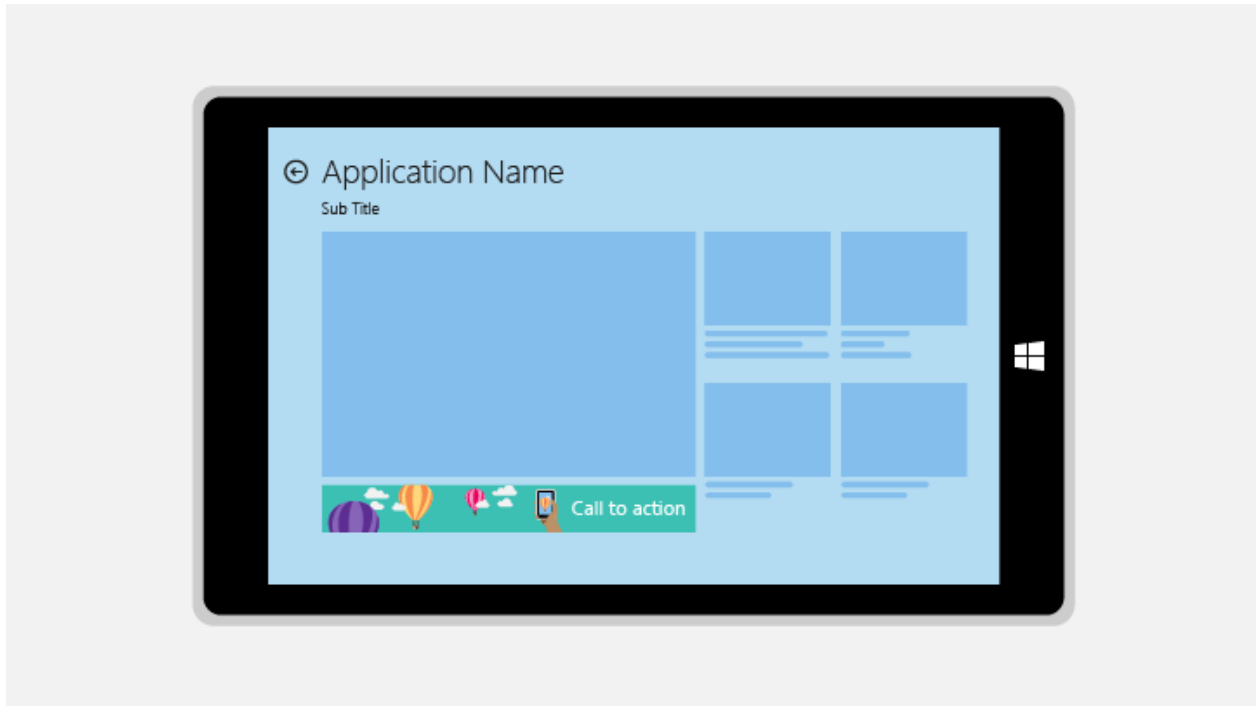


In-app purchases

Whether you charge a price for your app or offer it for free, you can use in-app purchases in your app to provide an ongoing revenue stream. Use in-app purchases to let customers upgrade from a free to a paid version of your app, or offer durable or consumable add-ons for sale within your app.

[Use in-app purchases](#)

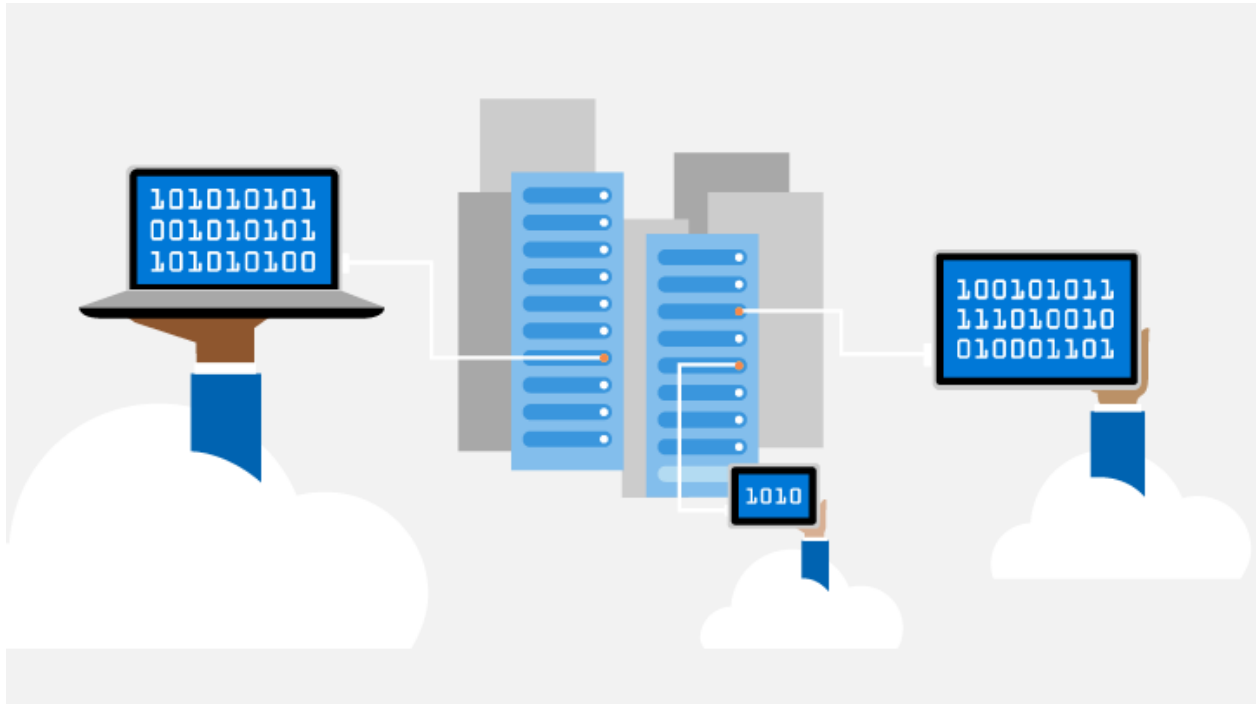
Monetize your app with ads



Ads for every context

We support a wide variety of ad experiences to suit most needs, including banner ads, interstitial ads (banner and video), linear video ads, playable ads, and native ads. Our platform is compliant with the OpenRTB, VAST 2.x, MRAID 2, and VPAID 3 standards and is compatible with MOAT and IAS.

[Explore ad options](#) [Install ad SDK](#)



Ad mediation service

Maximize ad revenue in your apps by using the microsoft ad mediation service to serve ads from multiple popular ad networks. You can configure your mediation settings in Partner Center without touching a line of code. If you let us configure mediation for you, our machine learning algorithms will help you maximize ad revenue across the markets your app supports.

[Use ad service](#)

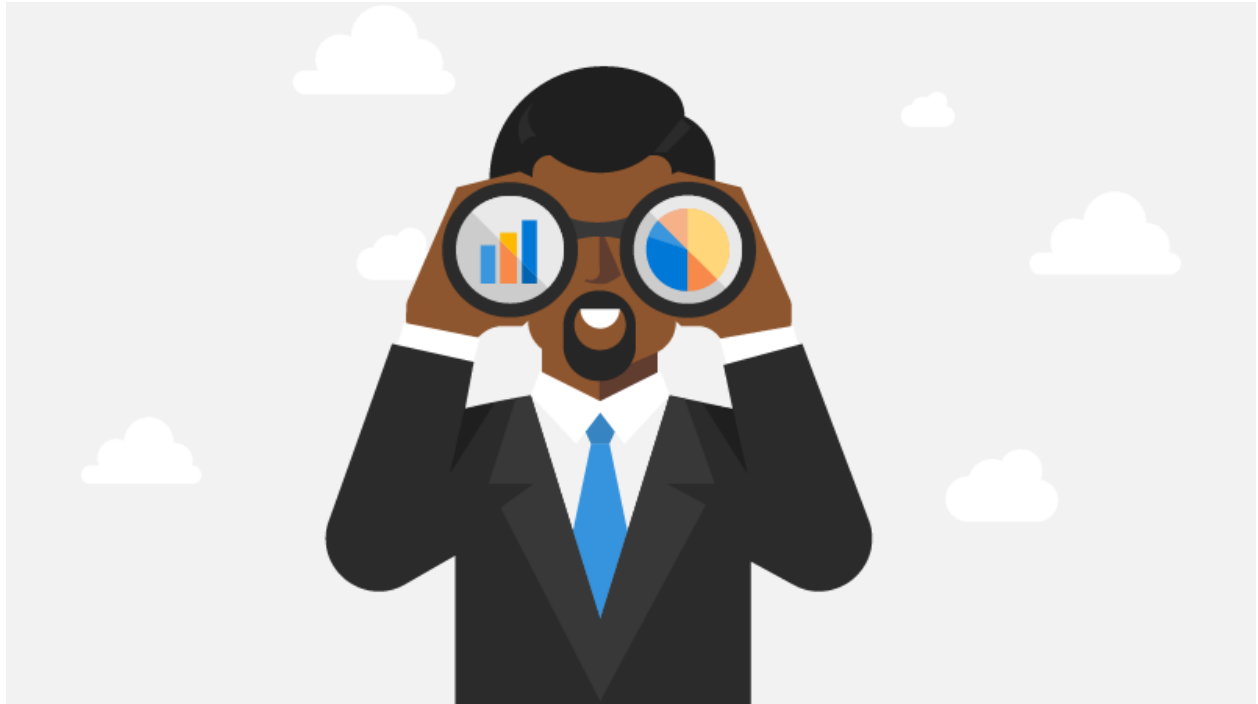


Analytics

Detailed analytics reports let you see how your ads in apps are performing, giving you the information you need to maximize your ad revenue. We also provide a RESTful API you can use to get this data programmatically.

[Review performance](#)

Other monetization opportunities



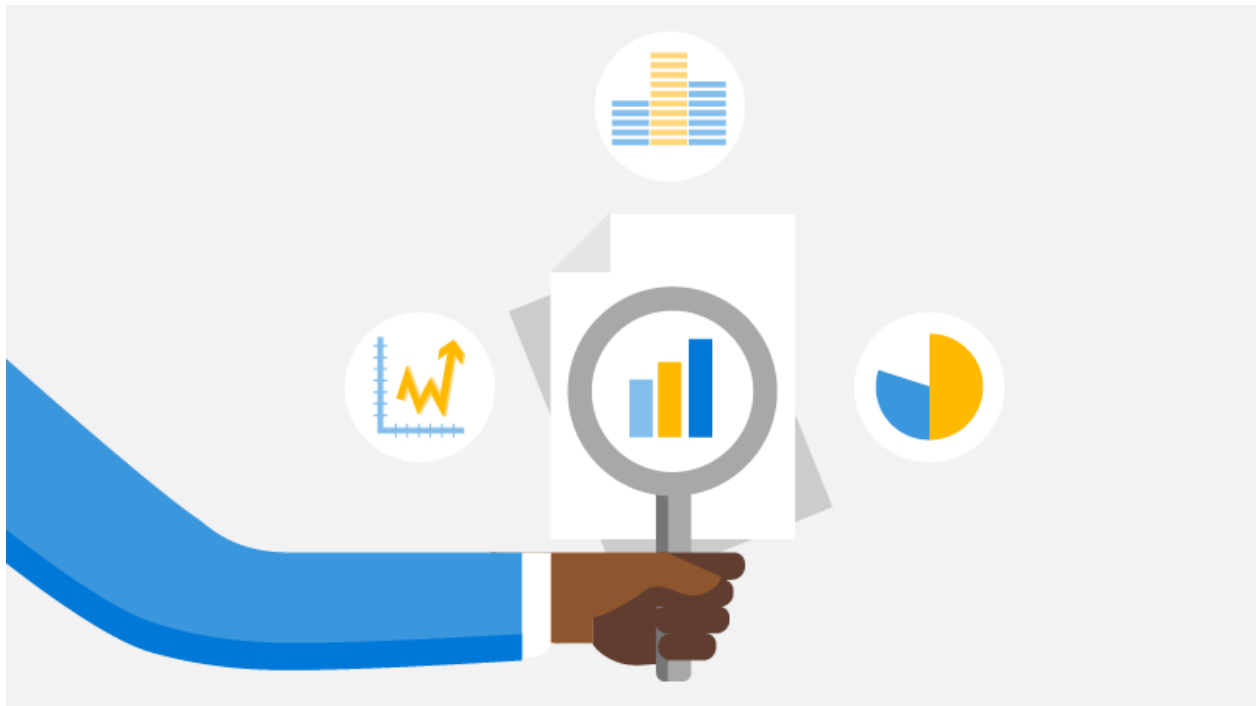
Looking for other ways to increase your monetization? Consider these options.

[Expand table](#)

Topic	Description
[Microsoft Affiliate Program] (https://www.microsoft.com/microsoft-365/business/microsoft-365-affiliate-program ↗)	Earn commissions by linking to Microsoft products from your app, blog, web page or other communications. You can link to apps, games, music, movies, hardware, accessories, and other goods sold in the Microsoft Store.
A/B experimentation	Run A/B tests in your apps to measure the effectiveness of feature changes on some customers before you enable the changes for everyone.
Engage customers with the Microsoft Store Services SDK	The Microsoft Store Services SDK provides libraries and tools that you can use to add features to your apps that help you engage with your customers. These features include

Topic	Description
	targeted notifications, A/B tests, and launching Feedback Hub from your app.
Launch Feedback Hub from your app	Add code to your UWP apps to direct your Windows 10 and Windows 11 customers to Feedback Hub, where they can submit problems, suggestions, and upvotes. Then, manage this feedback in the Feedback report in Partner Center. This feature requires the Microsoft Store Services SDK.
Configure your app to receive Partner Center push notifications	Register a notification channel for your UWP app so it can receive Partner Center push notifications , and track the rate of app launches that resulted from the push notifications. This feature requires the Microsoft Store Services SDK.
Log custom events for Partner Center	Log custom events from your UWP app and review the events in the Usage report in Partner Center. This feature requires the Microsoft Store Services SDK.
Request ratings and reviews	Encourage your customers to rate or review your app by programmatically showing a rating and review UI.
Microsoft Store services	Learn how to use RESTful APIs to automate submissions to the Store, access analytics data for your apps, and automate other tasks related to the Store.
Add retail demo (RDX) features to your app	Include a retail demo mode in your Windows app so customers who try out PCs and devices on the sales floor can jump right in.

Monetization analytics



Keep tabs on how your app is performing in the Store by using these reports.

- [Payout summary](#)
- [Acquisitions report](#)
- [Add-on acquisitions report](#)
- [Advertising performance report](#)
- [Get analytics data using our REST API](#)
- [Create customer segments](#)
- [Feedback report](#)
- [Usage report](#)

Packaging apps

Article • 10/20/2022

This section contains or links to articles about packaging Universal Windows Platform (UWP) apps in MSIX and .appx app packages for deployment and installation. Some of these links go to relevant articles in the [MSIX documentation](#).

📌 Note

The original app packaging format for UWP apps in Windows 10 was .appx. Starting in Windows 10, version 1809, this packaging format was renamed to .msix and extended to support all types of Windows apps, including .NET and C++ desktop (Win32) apps. Support for MSIX is also being extended to earlier Windows versions. For more information, see the [MSIX documentation](#).

Topic	Description
Package a UWP app with Visual Studio	To distribute or sell your Universal Windows Platform (UWP) app, you need to create an app package for it.
Manual app packaging	If you want to create and sign an app package, but you didn't use Visual Studio to develop your app, you'll need to use the manual app packaging tools.
App package architectures	Learn more about which processor architecture(s) you should use when building your app package.
UWP App Streaming Install	App Streaming Install enables you to specify which parts of your app you would like the Microsoft Store to download first. When the essential files of the app are downloaded first, the user can launch and interact with the app while the rest of it finishes downloading in the background.
Optional packages and related set authoring	Optional packages contain content that can be integrated with a main package. These are useful for downloadable content (DLC), dividing a large app for size restraints, or for shipping any additional content for separate from your original app.
Optional packages with executable code	Learn how to use Visual Studio to create an optional package with executable code.
Install Windows apps with App Installer	App Installer allows for Windows apps to be installed by double clicking the app package.

Topic	Description
Install apps with the WinAppDeployCmd.exe tool	Windows Application Deployment (WinAppDeployCmd.exe) is a command line tool that can use to deploy a UWP app from a Windows 10 machine to any Windows 10 Mobile device. You can use this tool to deploy an app package when the Windows 10 Mobile device is connected by USB or available on the same subnet without needing Microsoft Visual Studio or the solution for that app. This article describes how to install UWP apps using this tool.
Set up automated builds for your UWP app	If you want to package your app as part of an automated build process, this topic shows you how to use Visual Studio Team Services (VSTS) to do it.
App capability declarations	Capabilities must be declared in your app's package manifest to access certain API or resources like pictures, music, or devices like the camera or the microphone.
Download and install package updates from the Store	Your UWP app can programmatically check for package updates and install the updates. Your app can also query for packages that have been marked as mandatory in Partner Center and disable functionality until the mandatory update is installed.

App capability declarations

Article • 10/11/2023

ⓘ Note

Some information relates to pre-released product, which may be substantially modified before it's commercially released. Microsoft makes no warranties, express or implied, with respect to the information provided here.

Which kinds of apps do app capabilities apply to?

Most scenarios for app capabilities are relevant only to apps that have package identity, and that run in an AppContainer. All UWP apps meet those criteria; so capabilities apply to them. But you can also give a desktop app package identity, and configure it as an AppContainer app. So capabilities apply to some desktop apps, too.

- For more info about packaging and *package identity*, see [Deployment overview](#).
- For more info about AppContainer apps, see [AppContainer for legacy apps](#).

A desktop app that's an AppContainer app can be identified by

`uap10:TrustLevel="appContainer"` in its app package manifest (for more info, see [Application \(Windows 10\)](#)). Similarly, a desktop app with *mediumIL* (an integrity level of medium) has `uap10:TrustLevel="mediumIL"`. Medium IL apps—which are also known as *full trust apps*—don't run in an AppContainer.

The **internetClient** and **enterpriseAuthentication** capabilities grant an application the ability to perform certain operations that the user can already do. So those are examples of capabilities that apply only to AppContainer apps. Conversely, a Medium IL app is already running as the user; so an app like that can already perform those operations without requiring those capabilities.

But there are some scenarios where a Medium IL app should declare a capability, too. In fact, a Medium IL app *needs* to declare the **runFullTrust** restricted capability. And, to be able to register out-of-process COM servers for inter-process communication (IPC), a packaged app needs **runFullTrust**. That feature is known as *Packaged COM* (for more info, see the blog post [COM Server and OLE Document support for the Desktop Bridge](#) [↗](#)).

For info about another scenario that applies even to Medium IL apps, see [Privacy-sensitive capabilities](#) in this topic.

You can determine whether your app package manifest needs **runFullTrust** simply by building your package. `Makeappx.exe` will validate the schema, and if **runFullTrust** isn't declared but something needs it, then you'll see a detailed error message including what the problem is, together with line and column numbers.

Declaring capabilities

If you want to access certain APIs or resources (such as pictures or music), or devices (such as the camera or the microphone), then you must declare the appropriate app capabilities in your Windows app's [package manifest](#)

You can declare general capabilities by using the [Manifest Designer](#) in Visual Studio; or you can add them manually. For more info, see [How to specify capabilities in a package manifest](#). It's important to know that when customers get your app from the Microsoft Store, they're notified of all the capabilities that the app declares. So be sure to declare only the capabilities that your app needs.

Privacy-sensitive capabilities

A *sensitive resource* is a resource that can access the user's personal data, or cost the user money. In this topic, capabilities that provide apps with access to a *sensitive resource* are annotated by an asterisk (*) in the **Capability scenario** column.

Privacy-sensitive capabilities signal to the operating system (OS)—and to the user—what the app intends to do. Since it's good to send this signal to the users of your app, we recommend that you declare privacy-sensitive capabilities *even for Medium IL apps* (where the application identity is used to provide individual privacy toggles). Doing so allows those apps to be managed in the privacy settings pages (managed by the Windows **Settings** app) as soon as they're installed; as opposed to later, when they access privacy-sensitive resources.

Those privacy settings let the user dynamically control access to sensitive resources. Thus, it's important that your app doesn't assume that a sensitive resource is always available. For more info about accessing sensitive resources, see [Security](#).

Different kinds of capabilities

There are several kinds of capabilities.

- [General-use capabilities](#), which apply to most common app scenarios.
- [Device capabilities](#), which allow your app to access internal and peripheral devices.
- [Restricted capabilities](#), which require approval for Microsoft Store submission and/or are generally only available to Microsoft and certain partners.
- [Custom capabilities](#).

General-use capabilities

General-use capabilities are specified by using **Capability** elements in your app package manifest. These capabilities apply to the most common app scenarios.

ⓘ Note

All **Capability** elements must come before any [CustomCapability](#) and [DeviceCapability](#) elements under the **Capabilities** node in the package manifest.

 Expand table

Capability scenario	Capability usage
Music*	The musicLibrary capability provides programmatic access to the user's Music library, allowing the app to enumerate and access all files in the library without user interaction. This capability is typically used in jukebox apps that make use of the entire Music library. The file picker provides a robust UI mechanism that lets users open files for use with an app. Declare the musicLibrary capability only when the scenarios for your app require programmatic access and can't be realized by using the file picker. The musicLibrary capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="musicLibrary"/></Capabilities></pre>
Pictures*	The picturesLibrary capability provides programmatic access to the user's Pictures library, allowing the app to enumerate and access all files in the library without user interaction. This capability is typically used in photo apps that make use of the entire Pictures library. The file picker provides a robust UI mechanism that lets users open files for use with an app. Declare the picturesLibrary capability only when the scenarios for your app require programmatic access and can't be realized them by using the file picker.

Capability scenario	Capability usage
	The picturesLibrary capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="picturesLibrary"/></Capabilities></pre>
Videos*	The videosLibrary capability provides programmatic access to the user's Videos, allowing the app to enumerate and access all files in the library without user interaction. This capability is typically used in movie-playback apps that make use of the entire Videos library. The file picker provides a robust UI mechanism that lets users open files for use with an app. Declare the videosLibrary capability only when the scenarios for your app require programmatic access and can't be realized by using the file picker. The videosLibrary capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="videosLibrary"/></Capabilities></pre>
Removable Storage	The removableStorage capability provides programmatic access to files on removable storage, like USB keys and external hard drives, filtered to the file-type associations declared in the package manifest. For example, if a document-reader app declares a .doc file-type association, it can open .doc files on the removable storage device, but not other types of files. Be careful when you declare this capability, because users may include a variety of info in their removable storage devices, and will expect your app to provide a valid justification for programmatic access to the removable storage for all files of the declared type. Users will expect your app to handle any file associations that you declare. So don't declare file associations that your app cannot handle responsibly. The file picker provides a robust UI mechanism that lets users open files for use with an app. Declare the removableStorage capability only when the scenarios for your app require programmatic access and can't be realized by using the file picker. The removableStorage capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="removableStorage"/></Capabilities></pre>
Internet and public networks*	There are two capabilities that provide different levels of access to the Internet and public networks.

Capability scenario	Capability usage
	The internetClient capability indicates that apps can receive incoming data from the Internet. Cannot act as a server. No local network access. The internetClientServer capability indicates that apps can receive incoming data from the Internet. Can act as a server. No local network access. Most apps that have a web service component will use internetClient. Apps that enable peer-to-peer (P2P) scenarios where the app needs to listen for incoming network connections should use internetClientServer. The internetClientServer capability includes the access that the internetClient capability provides, so you don't need to specify internetClient when you specify internetClientServer.
Homes and work networks*	The privateNetworkClientServer capability provides inbound and outbound access to home and work networks through the firewall. This capability is typically used for games that communicate across the local area network (LAN), and for apps that share data across a variety of local devices. If your app specifies musicLibrary, picturesLibrary, or videosLibrary, you don't need to use this capability to access the corresponding library in a Home Group. On Windows, this capability does not provide access to the Internet.
Appointments	The appointments capability provides access to the user's appointment store. This capability allows read access to appointments obtained from the synced network accounts and to other apps that write to the appointment store. With this capability, your app can create new calendars and write appointments to calendars that it creates. The appointments capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="appointments"/></Capabilities></pre>
Contacts*	The contacts capability provides access to the aggregated view of the contacts from various contacts stores. This capability gives the app limited access (network permitting rules apply) to contacts that were synced from various networks and the local contact store. The contacts capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="contacts"/></Capabilities></pre>
Code generation	The codeGeneration capability allows apps to access the following functions which provide JIT capabilities to apps. VirtualProtectFromApp CreateFileMappingFromApp

Capability scenario	Capability usage
	OpenFileMappingFromApp MapViewOfFileFromApp
AllJoyn	The allJoyn capability allows AllJoyn-enabled apps and devices on a network to discover and interact with each other. All apps that access APIs in the Windows.Devices.AllJoyn namespace must use this capability.
Phone calls	The phoneCall capability allows apps to access all of the phone lines on the device and perform the following functions. • Place a call on the phone line and show the system dialer without prompting the user. • Access line-related metadata. • Access line-related triggers. • Allows the user-selected spam filter app to set and check block list and call origin information. The phoneCall capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="phoneCall"/></Capabilities></pre> The phoneCallHistoryPublic capability allows apps to read cellular and some VoIP call history information on the device. This capability also allows the app to write VoIP call history entries. This capability is required to access all members of the PhoneCallHistoryStore class.
Recorded Calls Folder*	The recordedCallsFolder device capability allows apps to access the recorded calls folder. The recordedCallsFolder capability must include the mobile namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><mobile:Capability Name="recordedCallsFolder"/> </Capabilities></pre>
User Account Information*	The userAccountInformation capability gives apps the ability to access the user's name and picture. This capability is required to access some APIs in the Windows.System.UserProfile namespace. The userAccountInformation capability must include the uap namespace when you declare it in your app's package manifest as shown below.

Capability scenario	Capability usage
	<pre><Capabilities><uap:Capability Name="userAccountInformation"/> </Capabilities></pre>
VoIP calling	The voipCall capability allows apps to access the VoIP calling APIs in the Windows.ApplicationModel.Calls namespace. The voipCall capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="voipCall"/></Capabilities></pre>
3D Objects	The objects3D capability allows apps to have programmatic access to the 3D object files. This capability is typically used in 3D apps and games that need access to the entire 3D objects library. This capability is required to access the folder that contains the 3D objects using APIs in the Windows.Storage namespace. The objects3D capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="objects3D"/></Capabilities></pre>
Chat Message Access*	The chat capability allows apps to read and delete SMS and MMS messages. It also allows apps to store messages in the system data store. This capability is required to access messages using APIs in the Windows.ApplicationModel.Chat namespace. The chat capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="chat"/></Capabilities></pre>
Read Blocked Messages*	The blockedChatMessages capability allows apps to read SMS and MMS messages that have been blocked by the Spam Filter app. This capability is required to access the blocked messages using APIs in the Windows.ApplicationModel.Chat namespace. The blockedChatMessages capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="blockedChatMessages"/></Capabilities></pre>
Custom Devices	The lowLevelDevices capability allows apps to access custom devices when a number of additional requirements are met. This capability should not be

Capability scenario	Capability usage
	confused with the lowLevel device capability, which allows access to GPIO, I2C, SPI, and PWM devices. If you develop a custom driver that exposes a device interface and you wish to open a handle to this device and send IOCTLs, you must: • Enable the lowLevelDevices capability in your application manifest: <pre><Capabilities><iot:Capability Name="lowLevelDevices"/> </Capabilities></pre> • Enable embedded mode • Mark the device interface as restricted, either in your INF or by calling WdfDeviceAssignInterfaceProperty() in your driver. You can then use Windows.Devices.Custom.CustomDevice to open a handle to your device. For more information, see UWP device apps for internal devices.
IoT System Administration	The systemManagement capability allows apps to have basic system administration privileges such as shutting down or rebooting, locale, and timezone. This capability is required to access some of the APIs in the Windows.System namespace. The systemManagement capability must include the iot namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><iot:Capability Name="systemManagement"/></Capabilities></pre>
Background Media Playback	The backgroundMediaPlayback capability changes the behavior of the media-specific APIs like the MediaPlayer and AudioGraph classes to enable media playback while your app is in the background. All active audio streams will no longer mute, but will continue to be audible when an app transitions to the background. Additionally, app lifetime will be extended automatically while playback is occurring.
Remote System	The remoteSystem capability allows apps to have access to a list of devices associated with the user's Microsoft Account. Access to the device list is necessary to perform any operations that persist across devices. This capability is required to access to all members of the following. • Windows.System.RemoteSystems namespace • Windows.System.RemoteLauncher class • AppServiceConnection.OpenRemoteAsync method
Spatial Perception	The spatialPerception capability provides programmatic access to spatial mapping data, giving mixed reality apps information about surfaces in application-specified regions of space near the user. Declare the

Capability scenario	Capability usage
	spatialPerception capability only when your app will explicitly use these surface meshes, as the capability is not required for mixed reality apps to perform holographic rendering based on the user's head pose.
Global Media Control	The globalMediaControl capability allows apps to access to playback sessions throughout the system that have integrated with SystemMediaTransportControls to provide playback info and allow remote control. This capability is required to use some APIs in the Windows.Media.Control namespace. This capability is defined in the uap7:Capability element.
Graphics Capture	The graphicsCapture capability allows apps to take screenshots of windows or displays when allowed by the user. This capability is required to use the Windows.Graphics.Capture.GraphicsCapturePicker object. This capability is defined in the uap6:Capability element.
Borderless Graphics Capture	The graphicsCaptureWithoutBorder capability allows apps to take screenshots without showing a screenshot border around the window or display. This capability is required to use the Windows.Graphics.Capture.GraphicsCaptureSession.IsBorderRequired property. This capability is defined in the uap11:Capability element.
Programmatic Graphics Capture	The graphicsCaptureProgrammatic capability allows apps to take screenshots of various windows or displays on its own. This capability is required to create a Windows.Graphics.Capture.GraphicsCaptureItem object from a WindowId or DisplayId . This capability is defined in the uap11:Capability element.
User Data Tasks	The userDataTasks capability provides access to APIs in the Windows.ApplicationModel.UserDataTasks namespace, which provide access to the task items that are stored in Windows by Exchange ActiveSync (EAS) connections and other provider apps.
User Notification Listener	The userNotificationListener capability provides access to APIs in the Windows.UI.Notifications.Management namespace, which enable management of user notifications.

Device capabilities

Device capabilities allow your app to access peripheral and internal devices. Device capabilities are specified by using **DeviceCapability** elements in your app package manifest. This element may require additional child elements and some device capabilities need to be added to the package manifest manually. For more info, see [How to specify device capabilities in a package manifest](#) and [DeviceCapability Schema reference](#).

ⓘ Note

You can have multiple **DeviceCapability** elements under the **Capabilities** element in the package manifest. All **DeviceCapability** elements must come after any **Capability** and [CustomCapability](#) elements.

 Expand table

Capability scenario	Capability usage
Location*	The location capability provides access to location functionality that is retrieved from dedicated hardware like a GPS sensor in the PC or is derived from available network info. Apps must handle the case in which the user has disabled location services from the Settings charm.
Microphone	The microphone capability provides access to the microphone's audio feed, which allows the app to record audio from connected microphones. Apps must handle the case in which the user has disabled the microphone from the Settings charm.
Proximity	The proximity capability enables multiple devices in close proximity to communicate with one another. This capability is typically used in casual multi-player games and in apps that exchange information. Devices attempt to use the communication technology that provides the best possible connection, including Bluetooth, Wi-Fi, and the Internet. This capability is used only to initiate communication between the devices.
Webcam	The webcam capability provides access to the video feed of a built-in camera or external webcam, which allows the app to capture photos and videos. On Windows, apps must handle the case in which the user has disabled the camera from the Settings charm. The webcam capability only grants access to the video stream. In order to grant access to the audio stream as well, the microphone capability must be added.
USB	The usb device capability enables access to APIs in the Updating the app manifest package for a USB device .
Human interface device (HID)	The humaninterfacedevice device capability enables access to APIs in the How to specify device capabilities for HID .
Point of Service (POS)	The pointOfService device capability enables access to APIs in the Windows.Devices.PointOfService namespace. This namespace lets your app access Point of Service (POS) barcode scanners and magnetic stripe readers. The namespace provides a vendor-neutral interface for accessing POS devices from various manufacturers from a UWP app.

Capability scenario	Capability usage
Bluetooth	The bluetooth device capability allows apps to communicate with already paired bluetooth devices over both Generic Attribute (GATT) or Classic Basic Rate (RFCOMM) protocol. This capability is required to use some APIs in the Windows.Devices.Bluetooth namespace.
Wi-Fi Networking	IMPORTANT. The wiFiControl device capability will be affected by upcoming changes to operating system behavior, planned for fall 2024. For more info, see Changes to API behavior for Wi-Fi access and location. The wiFiControl device capability allows apps to scan and connect to Wi-Fi networks. This capability is required to use some APIs in the Windows.Devices.WiFi namespace.
Radio state	The radios device capability allows apps to toggle the Wi-Fi and Bluetooth radios. This capability is required to use the APIs in the Windows.Devices.Radios namespace.
Optical disc	The optical device capability allows apps to access functions on optical disk drives such as CD, DVD, and Blu-ray. This capability is required to use some APIs in the Windows.Devices.Custom namespace.
Motion activity	The activity device capability allows apps to detect the current motion of the device. This capability is required to use some APIs in the Windows.Devices.Sensors namespace.
Presence Sensing	The humanPresence device capability allows apps to access Presence Sensors on the device which can provide information on user presence and engagement. This capability is required to use some APIs in the Windows.Devices.Sensors namespace.
Serial communication	The serialcommunication device capability provides access to APIs in the Windows.Devices.SerialCommunication namespace, which allows a Windows app to communicate with a device that exposes a serial port or some abstraction of a serial port. This capability is required to use the APIs in the Windows.Devices.SerialCommnication namespace.
Eye Tracker	The gazeInput capability allows apps to detect where the user is looking within the application bounds when a compatible eye tracking device is connected or for Mixed Reality devices that support gaze tracking. This capability is required to use some APIs in the Windows.Devices.Input.Preview

Capability scenario	Capability usage
	namespace. For Mixed Reality devices, this capability is required for APIs in the Windows.Perception.People.EyesPose .
GPIO, I2C, SPI, and PWM	The lowLevel device capability provides access to GPIO, I2C, SPI, and PWM devices. This capability is required to use the APIs in the following namespaces: Windows.Devices.Gpio, Windows.Devices.I2c, Windows.Devices.Spi, Windows.Devices.Pwm. <pre><Capabilities><DeviceCapability Name="lowLevel"/></Capabilities></pre>

Restricted capabilities

If your app declares any restricted capabilities, then you must provide info during the [app submission process](#) in order to be approved to publish your app to the Microsoft Store. You provide this info on the [Submission options](#) page of your submission, explaining how your app uses each restricted capability that it declares.

Important

Restricted capabilities are intended for very specific scenarios. The use of these capabilities is highly restricted and subject to additional Store onboarding policy and review. Note that you can sideload apps that declare restricted capabilities without needing to receive any approval. Approval is only required when submitting these apps to the Store.

Be sure not to declare these restricted capabilities unless your app truly needs them. There are cases where such capabilities are necessary and appropriate, such as banking with two-factor authentication, where users provide a smart card with a digital certificate that confirms their identity. Other apps may be designed primarily for enterprise customers and may need access to corporate resources that can't be accessed without the user's domain credentials.

To declare a restricted capability, modify your [app package manifest](#) source file (`Package.appxmanifest`). Add the **xmlns:rescap** XML namespace declaration, and use the **rescap** prefix when you declare your restricted capability. For example, here's how to declare the **appCaptureSettings** capability.

XML


```
<?xml version="1.0" encoding="utf-8"?>
<Package
    ...

xmlns:rescap="http://schemas.microsoft.com/appx/manifest/foundation/windows1
0/restrictedcapabilities"
    IgnorableNamespaces="... rescap">
    ...
<Capabilities>
    <rescap:Capability Name="appCaptureSettings"/>
</Capabilities>
</Package>
```

ⓘ Note

All restricted capability elements must come before any CustomCapability and DeviceCapability elements under the **Capabilities** node in the package manifest.

Restricted capability approval process

Previously, we required you to contact support to get approval to use a capability. We now allow you to provide this info in [Partner Center](#) as part of the [submission process](#).

When you upload packages for your submission, we will detect whether any restricted capabilities are declared. If we do so, you will be required to provide details about how your product uses each capability on the [Submission options](#) page. Be sure to provide as much detail as possible to help us understand why your product needs to declare the capability. Note that this may add some additional time for your submission to complete the certification process.

During the certification process, our testers will review the info you provide to determine whether your submission is approved to use the capability. Note that this may add some additional time for your submission to complete the certification process. If we approve your use of the capability, your app will continue through the rest of the certification process. You generally will not have to repeat the capability approval process when you submit updates to your app (unless you declare additional capabilities).

If we don't approve your use of the capability, your submission will fail certification, and we will provide feedback in the certification report. You then have the option to create a new submission and upload packages which don't declare the capability, or, if applicable, address any issues related to your use of the capability and request approval in a new submission.

ⓘ Note

If your submission uses a development sandbox in Partner Center (for example, this is the case for any game that integrates with Xbox Live), you must request approval in advance rather than providing info on the **Submission options** page. You can request approval in advance if you're publishing a game to Xbox through [ID@Xbox](#) or as a managed partner. To do so, please contact your Microsoft account team.

The team will need a brief description about how you are using the capability and why it is necessary for your product. If you do not provide all the information necessary, your request will be denied. You may also be asked to provide more information. Note that this process typically takes 5 business days or longer, so please submit your request well in advance.

You may also use this method of requesting approval (rather than providing this info during your submission), whether or not you're using a development sandbox, if you prefer to confirm that you are approved to use a restricted capability before you start your submission.

Restricted capability list

The following table lists the restricted capabilities. You may request approval for these capabilities in apps that you submit to the Store by following the process described above.

ⓘ Important

Some of these restricted capabilities are almost never approved for apps submitted to the Store, except in very specific and limited circumstances. These capabilities are called out in the table below. We recommend not declaring these capabilities in your app if you plan to distribute it through the Store.

 Expand table

Capability scenario	Capability usage
Enterprise	Windows domain credentials enable a user to log into remote resources using their credentials, and act as if a user provided their user name and password. The enterpriseAuthentication capability is typically used in line-of-business

Capability scenario	Capability usage
	apps that connect to servers within an enterprise. You don't need this capability for generic communication across the Internet. The enterpriseAuthentication capability is intended to support common line-of-business apps. Don't declare it in apps that don't need to access corporate resources. The file picker provides a robust UI mechanism that enables users to open files on a network share for use with an app. Declare the enterpriseAuthentication capability only when the scenarios for your app require programmatic access, and you cannot realize them by using the file picker. The enterpriseAuthentication capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="enterpriseAuthentication"/> </Capabilities></pre> This capability is required to call the GetUserNameEx function. The enterpriseDataPolicy capability allows apps to handle enterprise data separately and safely when the app is managed with Windows Information Protection policy (For example: Mobile Device Management and Mobile Application Management systems). Declare this restricted capability as shown below. <pre><Capabilities><rescap:Capability Name="enterpriseDataPolicy"/> </Capabilities></pre> This capability is required to use all members of the following classes. • FileProtectionManager • DataProtectionManager • ProtectionPolicyManager
Shared user certificates	The sharedUserCertificates capability enables an app to add and access software and hardware-based certificates in the Shared User store, such as certificates stored on a smart card. This capability is typically used for financial or enterprise apps that require a smart card for authentication. The sharedUserCertificates capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="sharedUserCertificates"/> </Capabilities></pre>

Capability scenario	Capability usage
Documents*	The documentsLibrary capability provides programmatic access to the user's Documents library, filtered to the file type associations declared in the package manifest. For example, if a word processing app declared a .doc file type association, it can open .doc files in the user's Documents library. The documentsLibrary capability is <i>only</i> needed if your application programmatically accesses the Documents library <i>without</i> user intervention. Your application does not need the documentsLibrary capability to access the Documents library if the user chooses it with a picker API. Generally, apps should allow the user to choose the location of their files, using one of the following picker APIs: • FileOpenPicker to open an existing file. • FileSavePicker to save a new file. • FolderPicker to choose a folder to open / save additional files from. Using these APIs allows the user to choose a location that works best for them, such as a cloud-synced account (eg, OneDrive). After the user has picked a file or folder using these APIs, your app can get ongoing access to the location by using the FutureAccessList API. This API allows your app to access the files or folders in the future without asking the user to pick them again. In cases where existing workflows assume files will be in the Documents library (for example, interop with an existing desktop application) or where you do not want the user to have to choose the location, you can declare the documentsLibrary capability for your application. If you use the documentsLibrary capability for your application, it is recommended that you also allow the user to pick locations manually. The documentsLibrary capability must include the uap namespace when you declare it in your app's package manifest as shown below. <pre><Capabilities><uap:Capability Name="documentsLibrary"/></Capabilities></pre>
Game DVR Settings	The appCaptureSettings restricted capability allows apps to control the user settings for the Game DVR. This capability is required to use some APIs in the Windows.Media.Capture namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Cellular	The cellularDeviceControl restricted capability allows apps to have control over the cellular device.

Capability scenario	Capability usage
	The cellularDeviceIdentity capability allows apps to access cellular identification data. The cellularMessaging capability allows apps to make use of SMS and RCS. These capabilities are required to use some APIs in the Windows.Devices.Sms namespaces.
Device Unlock	The deviceUnlock restricted capability allows apps to unlock a device for developer and enterprise sideloading scenarios. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Dual SIM Tiles	The dualSimTiles restricted capability allows apps to create an additional app list entry on devices that have multiple SIMs. This capability is required to use some APIs in the Windows.UI.StartScreen namespace.
Enterprise Shared Storage	The enterpriseDeviceLockdown restricted capability allows apps to use the device lock down API and access the enterprise shared storage folders. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
System Input Injection	The inputInjectionBrokered restricted capability allows apps to inject various forms of input such as HID, touch, pen, keyboard or mouse into the system programmatically. This capability is typically used for collaboration apps that can take control of the system. For a PC, input injection from an app that has this capability will only be received by processes in the same App Container. <pre><Capabilities><rescap:Capability Name="inputInjectionBrokered" /> </Capabilities></pre>
Observe Input*	The inputObservation restricted capability allows apps to observe various forms of raw input such as HID, touch, pen, keyboard, or mouse being received by the system regardless of its final destination. This capability and the APIs related to it are only available for use by select Microsoft partners.
Suppress Input	The inputSuppression restricted capability allows apps to suppress various forms of raw input such as HID, touch, pen, keyboard, or mouse from being

Capability scenario	Capability usage
	received by the system. This capability and the APIs related to it are only available for use by select Microsoft partners.
VPN App	The networkingVpnProvider restricted capability allows apps to have full access to VPN features, including the ability to manage connections and provide VPN Plugin functionality. This capability is required to use some APIs in the Windows.Networking.Vpn namespace.
Other App Management	The packageManagement restricted capability allows apps to manage other apps directly. The packageQuery device capability allows apps to gather information about other apps. These capabilities are required to access some methods and properties in the PackageManager class.
Screen Projection	The screenDuplication restricted capability allows apps to project the screen on another device. This capability is required to use APIs in the DirectX namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
User Principal Name	The userPrincipalName restricted capability allows apps to access the user principal name (UPN) of the current user. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Wallet	The walletSystem restricted capability allows apps to have full access to the stored wallet cards. This capability is required to use APIs in the Windows.ApplicationModel.Wallet.System namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.

Capability scenario	Capability usage
Location History	The locationHistory restricted capability allows apps to access the location history of the device. This capability is required to use APIs in the Windows.Devices.Geolocation namespace.
App Close Confirmation	The confirmAppClose restricted capability allows apps to close themselves, their own windows, and delay the closing of their app. Apps may request this capability in Windows 10 version 1703 (build 10.0.15063) and beyond. In prior Windows 10 versions the capability is private and will cause app install to fail with error message "The requested capability can not be authorized for this application."
Call History*	The phoneCallHistory restricted capability allows apps to read the call history and to delete entries in the history. This capability is required to use APIs in the Windows.ApplicationModel.Chat namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
System Level Appointment Access	The appointmentsSystem restricted capability allows apps to read and modify all appointments on the user's calendar. This capability is required to use APIs in the Windows.ApplicationModel.Appointment namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
System Level Chat Message Access*	The chatSystem restricted capability allows apps to read and write all SMS and MMS messages. This capability is required to use APIs in the Windows.ApplicationModel.Chat namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
System Level Contact Access	The contactsSystem restricted capability allows apps to read contact information that has been designated as restricted or sensitive and modify existing contact information.

Capability scenario	Capability usage
	This capability is required to use APIs in the Windows.ApplicationModel.Chat namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Email Access	The email restricted capability allows apps to read, triage, and send user emails. This capability is required to use APIs in the Windows.ApplicationModel.Email namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
System Level Email Access	The emailSystem restricted capability allows apps to read, triage, and send user restricted or sensitive emails. This capability is required to use APIs in the Windows.ApplicationModel.Email namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
System Level Call History Access	The phoneCallHistorySystem restricted capability allows apps to fully modify the call history by changing existing entries and writing new ones. This capability is required to use APIs in the Windows.ApplicationModel.Calls namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Send Text Messages*	The smsSend restricted capability allows apps to send SMS and MMS messages. This capability is required to use APIs in the Windows.ApplicationModel.Chat namespace.
System Level Access to All User Data	The userDataSystem restricted capability allows apps to access the user data system datastore. We don't recommend that you declare this capability in applications that you

Capability scenario	Capability usage
	submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Store Preview Features	The previewStore restricted capability allows apps to retrieve and purchase SKUs of in-app products. This capability is required to use certain APIs in the Windows.ApplicationModel.Store.Preview namespace.
First-Time Sign-in Settings	The firstSignInSettings restricted capability allows apps to access user settings that were set when the user first signed in to their device.
Windows Team Experience	The teamEditionExperience restricted capability allows apps to access internal APIs that control many experiential aspects of a Windows Team session. A Windows Team session is likely to be running on a team device such as a Microsoft Surface Hub. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Remote Unlock	The remotePassportAuthentication restricted capability allows apps to access credentials that can be used to unlock a remote PC. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Preview Composition	The previewUiComposition restricted capability allows apps to preview the Windows.UI.Composition namespace for their user interface so they can provide feedback on the API before it is completed. Please contact wincomposition@microsoft.com for more information.
Secure Assessment Lockdown	The secureAssessment restricted capability allows apps to lockdown Windows into a single app mode for secure assessments. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Connection Manager Provisioning	The networkConnectionManagerProvisioning restricted capability allows apps to define the policies that connect the device with WWAN and WLAN interfaces. Apps that use this capability are created by Mobile Operators to govern the devices that connect to their mobile network.
Data Plan Provisioning	The networkDataPlanProvisioning restricted capability allows apps to gather information about data plans on the device and read network usage. Apps that

Capability scenario	Capability usage
	use this capability are created by Mobile Operators to integrate their customers' actual data usage into the OS Data usage setting.
Software Licensing	The slapiQueryLicenseValue restricted capability allows apps to query software licensing policies. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Extended Execution	The extendedBackgroundTaskTime restricted capability prevents background tasks from being cancelled or terminated due to execution time limits. They are still subject to all other memory and energy usage limits. This capability can be restricted using the Battery Usage or Privacy Background Apps Settings. Note that consumers and administrators still have the ability to control background tasks through the Group Policy settings. The extendedExecutionBackgroundAudio restricted capability allows apps to play audio when the app is not in the foreground. The extendedExecutionCritical restricted capability allows apps to begin a critical extended execution session. The extendedExecutionUnconstrained restricted capability allows apps to begin an unconstrained extended execution session. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved. See Postpone app suspension with extended execution for more information about using extended execution to postpone when your app is suspended.
Mobile Device Management	The deviceManagementDmAccount restricted capability allows apps to provision and configure Mobile Operator Open Mobile Alliance - Device Management (MO OMA-DM) accounts. The deviceManagementFoundation restricted capability allows apps to have basic access to the Mobile Device Management (MDM) configuration service provider (CSP) infrastructure on the device. Note that other capabilities are needed to access specific CSPs. The deviceManagementWapSecurityPolicies restricted capability allows apps to configure Wireless Application Protocol (WAP)-based services such as MMs, Service Indication/Service Loading (SI/SL), and Open Mobile Alliance - Client Provisioning (OMA-CP).

Capability scenario	Capability usage
	The deviceManagementEmailAccount restricted capability allows apps created by Mobile Operators to add and manage an email account on devices they provision to users.
Package Policy Control	The packagePolicySystem restricted capability allows apps to have control of system policies related to apps that are installed on the device. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Games List	The gameList restricted capability allows apps to get a list of known games installed on the system. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Xbox Accessory	The xboxAccessoryManagement restricted capability allows apps to directly manage Xbox devices that conform to the Xbox hardware specification. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Speech Recognition for Accessories	The cortanaSpeechAccessory restricted capability allows apps to invoke and pass commands to Cortana. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Accessory Management	The accessoryManager restricted capability allows apps to register as an accessory app and opt-in to specific app notifications so that they may be forwarded to accessories and display to the user.
Driver access	The interopServices restricted capability allows apps to interact directly with drivers. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Foreground observation	The inputForegroundObservation restricted capability allows apps in the foreground to intercept keyboard input and byasses all non-app keyboard input processing. SAS combinations cannot be intercepted by this capability. This capability is required to access members of the KeyboardDeliveryInterceptor class.

Capability scenario	Capability usage
OEM and MO Partner apps	The oemDeployment restricted capability allows apps that are created by Microsoft partners to install new apps and query currently installed apps on the device. The oemPublicDirectory restricted capability allows apps that are created by Microsoft partners to have access to the shared app folder. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
App Licensing	The appLicensing restricted capability allows apps to run without the need of a license. You cannot submit your app to the store if you declare this capability in your manifest. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Location System	The locationSystem restricted capability allows apps to perform certain privileged location configurations like setting the default location for the device. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
User Data Accounts Provider	The userDataAccountsProvider restricted capability allows apps to fully manage the mail, calendar, and contact accounts.
Pen Workspace	The previewPenWorkspace capability allows an app to access the <code>Windows.ApplicationModel.Preview.Notes</code> namespace to be hosted inside the pen workspace as the remember action handler.
Secondary Authentication Factor	The secondaryAuthenticationFactor capability allows an app to unlock a PC by passing the secrets store on a nearby companion authentication device. For example, a companion fitness band can be used to unlock the PC. This capability is required to access APIs in the <code>Windows.Security.Authentication.Identity.Provider</code> namespace. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Store License Management	The storeLicenseManagement capability allows Microsoft partner hub-apps to manage store licenses on the device. This capability is required to access APIs in the <code>Windows.ApplicationModel.Store.LicenseManagement</code> namespace.

Capability scenario	Capability usage
User System ID	The userSystemId capability allows apps to get a system identifier specific to the user. This identifier uniquely identifies the current user on a specific system and can be used to correlate information across apps. This capability is required to access the SystemIdentification.GetSystemIdForUser(User) method.
Targeted Content	The targetedContent capability provides an application the ability to retrieve and use targeted subscription content provided by the Windows.Services.TargetedContent namespace. This capability is required to use some APIs in the Windows.System.Profile.SystemIdentification namespace.
UI Automation	The uiAutomation capability allows a UI automation client, such as Narrator, to connect to a UI Automation server or provider. This capability is required to use some APIs in the Windows.Xbox.Media.Capture.Broadcaster namespace.
Game Bar Services	The gameBarServices is restricted to 1st party store updatable inbox UWAs. This capability is required to use the Windows.Media.Capture.GameBarsServices class. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
App Capture Services	The appCaptureServices capacity is limited to parties with which Microsoft has contractual relationships. These relationships are granted based on partner agreements, which are being driven with the help of Xbox Services and bizdev. This capability is required to use the Windows.Media.Capture.AppCaptureServices class.
App Broadcast Services	The appBroadcastServices capability is limited to parties with which Microsoft has contractual relationships. These relationships are granted based on partner agreements, which are being driven with the help of Xbox Services. This capability is required to use the Windows.Media.capture.AppBroadcastServices class. We don't recommend that you declare this capability in applications that you submit to the Microsoft Store. In most cases, the use of this capability won't be approved.
Audio Device Configuration	The audioDeviceConfiguration This capability allows an application to query, configure, enable, and disable audio effects exposed by the audio driver.